

Q-Framework 開發與驗證白皮書 v1.0 RC2

異構 AI 基礎設施中的運算狀態虛擬化、自適應流量控制與可驗證跨裝置續算

2026 年 10 月 7 日

本白皮書記錄的是一套正在實際運行、並保留 SHA/receipt/runtime evidence 的系統。

Q-Framework 是一套實驗性的異構 AI 運算架構。研究核心不是把多張 GPU 假裝成一張巨大 GPU，而是把 **compute state**、**identity state**、**continuity state** 與 **execution lineage** 從單一 GPU 的生命週期中抽離，使工作可以被 Freeze、Hash、Move、Restore、Resume、Branch 與再次驗證。

RC2 依據 CURRENT 的實際開發與 Smoke/runtime 紀錄，補入四條先前在 RC1 中描述不足、但已具有實際運算證據的核心路徑。

1. 核心設計原則

1. Logical state 與 physical residency 分離。
2. GPU 是 execution worker，不是 workload state 的唯一 owner。
3. Transport integrity 與 compute progression 必須分開驗證。
4. 同一 lineage 可以續算，也可以從共同 root 分支執行不同 action/shot。
5. 所有公開主張都必須對應 runtime、SHA、receipt 或可量測結果。

2. Q-Framework 四大已驗證運算路徑

2.1 混卡運算 — Mixed-GPU Stateful Continuation

混卡運算不是把四張 GPU 的 VRAM 相加，而是讓同一條計算 lineage 在不同 GPU 之間交棒。

已驗證的底層 F-Block XCOPY：

- GPU0 VRAM → CPU → SharedMemory / Bridge → GPU1 VRAM。
- Block 大小：262,144,000 bytes，約 250 MiB。
- source SHA256、Bridge RAM SHA256、target GPU read-back SHA256 全部一致：
330674e23bf109a4f5e3983608ce118274373deae669e3d3bacee2f77639d5a6
- bitwise_sha_match=true

- sentinel_match=true
- source state 在 restore 前可被釋放。
- 實測總 round-trip 約 3.6753 秒。

更高層的 H3 rescue 也已跑通：

- seed 2609282350
- B0 checkpoint → CPU/RAM Bridge → B1 restore → sampler continuation → MP4 output
- output SHA256：307828DE5D0D54F36004EF9C97B2E4DAE5178E59A9C8EF74775F07522B788AB3

這證明 Q-Framework 的跨 GPU 路徑不是單純複製媒體檔，而是可搬運、可驗證並可繼續計算的 state lineage。

Progressive State Block

同一 lineage 在每次 GPU 接手並真正推進 step 後，重新 Freeze 會得到新的 QREV / SHA。這是預期行為：

snapshot_n → target GPU → compute → snapshot_{n+1}

例如 seed 2609292003 的紀錄中，A0／B1 之間持續 restore、compute、freeze，next_step 從 s02、s04、s06、s08 一路推進到後續 step；有實際 compute progression 時 SHA 隨 revision 改變，而沒有推進的 s00 HARD_OOM 控制組則維持相同 SHA。

因此：

傳輸時 SHA 相同 = state integrity。

續算後 SHA 改變 = compute progression。

兩者不能混為一談。

2.2 同卡續算 — Fixed-Owner Same-GPU Resume

Q-Framework 也保留「同一 chain 固定在同一 GPU owner」的模式：

qmrna_vvram_same_gpu

此模式的核心不是單卡獨立重跑，而是：

- 每條 story chain 有固定 owner slot。
- same_gpu_resume=true。
- chain 內保持 stable seed / lineage。
- previous GPU tail、QmRNA checkpoint、result receipt 與 Dynamic Static 可延續下一段。
- 四條 chain 可以平行存在，但每一條 chain 優先由自己的 owner GPU 續算。

為避免四條 chain 生成四個不同人物 root，CURRENT 已加入：

- 共用 deterministic CHARACTER_ROOT seed。

- H3 ref0 永久固定為 Character Master。
- ref1 用於 selected Static／previous tail／continuity。
- Character Master embedding 依 SHA-256 建立一次後供多卡、多段共用。

這個模式的價值在於建立 **GPU-local continuity**：當沒有必要跨卡時，不需為了調度而搬運 state；當 owner GPU 不可用或發生 PRE_OOM/QREV rescue 時，才進入跨卡恢復路徑。

白皮書將它與 mixed-GPU path 分開，是因為兩者解決不同問題：

- Same-GPU Resume：減少不必要 migration。
- Mixed-GPU Resume：提升可用性、彈性與 failover。

2.3 H3 Memory Static / QSTATIC — Shared State, Identity and Dynamic Static Compute

QSTATIC 不只是影片素材庫。

在 Q-Framework 中，Static 可以承擔 motion／pose／scene／composition／lighting 等 visual state；Character Master 則維持 identity authority。CPU/QWorker 可以從上一個 GPU 真實輸出建立下一段 Dynamic Static contract，再交給下一個 GPU。

H3 State Block Signature Mapping

CURRENT 已對 H3 的 reference state 做可重現 SHA mapping。

Audio Reference Block

- shape = [1, 32, 2, 40]
- raw bytes = 10,240
- 相同輸入重算 SHA 完全相同。
- 不同聲音內容在 shape 相同時產生不同 SHA。

Visual Reference Block

- 480×832 reference latent
- shape = [1, 24, 1, 52, 30]
- raw bytes = 149,760
- base repeat SHA 完全相同。
- 改變輸入內容後 shape 不變，但 SHA 改變。

Face Reference Visual Block — 跨 Node

同一 512×512 face reference 在 Node A/A1 與 Node B/B1 使用相同 H3 Video VAE 時：

- shape = [1, 24, 1, 32, 32]
- raw bytes = 98,304

- Node A / Node B SHA 完全一致：

1488ccf08323200643d5759046f2d47e1e16d44a70f278a79346aec53103d937

hflip variant 在兩個 Node 也得到相同 SHA。

這是 **environment-decoupled state** 的直接證據：至少在已測試的 H3 reference encode block 中，兩個不同節點不需要整機環境完全一致，仍能得到 bit-identical visual latent state。

Dynamic Static Production Loop

Job268 的 QRS0001 已實際完成：

- 24 frames / 1s context
- QmRNA checkpoint 在 B1 ↔ A1 間推進到 COMPLETE
- Face QC PASS
- median similarity = 0.310109
- min similarity = 0.245260
- threshold = 0.22

其後系統產生：

- result_QRS0001.json
- handoff_QRS0001_TAIL.png
- dynamic_static_QRS0002.json
- computed_by = QW9
- schema = hf-dynamic-static-v1
- image authority = PREVIOUS_GPU_TAIL
- identity authority = USER_UPLOAD

接著 QRS0002 才被 dispatch。

因此「上一手 GPU 真實輸出 → QmRNA → CPU STATIC_EVOLVE → 新 Dynamic Static → 下一手 GPU」已經進入真實 production flow，而不是只存在於設計文件。

四 GPU 共享人物狀態

QSTATIC / Character Master 架構也完成過四 Worker smoke。共同 identity / continuity state 在 A0 / A1 / B0 / B1 分支前以 SHA 驗證一致；之後各卡可以接不同 action / shot。

這裡需要保持一致的是 **共享 root / authoritative state**，不是不同動作完成後的最終影片 SHA。

2.4 混卡跨檔運算 — Cross-File / Cross-Shot Stateful Compute

第四條路徑是跨「檔案 / Shot / Boundary」的 stateful compute。

傳統影片串接通常是：

Shot A → decode RGB → final frame → Shot B re-encode

Q-Framework 的 Shot XCOPY / Motion Bridge 方向是：

Shot A state → Boundary Memory / QBlock → Bridge → Shot B / Solver

真正搬運與使用的是可延續 state，而不只是 RGB 截圖。

64 MiB Cross-Host Round Trip

CURRENT 已驗證：

B0 → Bridge → A1 GPU compute x+7 → Bridge → B0 restore

raw state SHA 與 sentinel values 經驗證。這代表 state 不只跨主機傳輸，而且真的在另一張 GPU 上執行運算後，再回傳形成新的 state。

Motion Bridge：不同 Boundary 檔案共同求解

Job #135 / X1 使用：

- RS01 tail reference SHA：
D539AD5EAEA6B5BED4132A94CD24689AB4D4AD61142582894BF6DAD93290ED6B
- RS02 head reference SHA：
1E51DBAAF8D30BAEBDB98D8393E82221FF1EEADAB93E2D9A50A7C100CFC444F4

MiniMax H3 接受兩個 boundary reference，進入真正 ReferenceToVideo conditioning；X1 smoke 經：

B0 → PRE-00M → A0 restore → H3 output

輸出：

- 480×960
- 5 frames
- SHA256：7A909C59636C1A18C6B1A1C7CD2037120F25DDC2C7F36CD2E62B4C70A43212E1

後續 R3 lineage：

B0 s01 → A0 s02 → B1 s04 → A0 s06 → B1 s08 → A0 s10 → A0 s12 → B1 s14 → A0 s16 → B1 s18 → A0 s20 → B1 final

完成 final artifact：

- SHA256：C35FAECFE3C1BD0B5338CC4BE0EB8A854A10CE0FEEF86D39CA750DE6BB4A357C

這就是 跨 Shot／跨檔 Boundary Memory + 跨 GPU Compute Continuation 的實際案例。

3. 節點 + GPU 運算一致性

Q-Framework 還有一條與四大路徑交叉的驗證：**跨執行環境 solver consistency**。

HF_QWORKER_LITE V0.1 使用真實 production sampler state，讓：

- Utility01：無 ComfyUI、無 PyTorch math path
- H410：無 ComfyUI math path
- Node B：PyTorch reference

三個環境重新執行同一段 production sampler arithmetic。

結果三邊的 recomputed-x SHA **完全相同**：

330271d14ee7086e27f875a4a56e720f169cdef1523e908ad94696bb70df15a4

相對於 source state 的 reverse→forward reconstruction 誤差：

- max abs error = $4.76837158203125e-07$
- mean abs error $\approx 2.03e-08$
- exact elements = 523,127 / 740,352
- exact ratio ≈ 0.706592

這裡的正確解讀是：

- 三個不同執行環境得到相同 **recomputed result SHA**。
- 相對 source 的極小差異來自 float32 reverse→forward rounding。
- 這不是宣稱任何任意 GPU workload 都 universal bitwise deterministic。
- 它證明真實 production state 可以脫離原 ComfyUI process，由另一個 node/runtime solver 重現一致的運算結果。

Node + GPU computation fingerprint near-match

除了上述三個 runtime 的 **recomputed-x SHA 完全一致**之外，四 Worker / QSTATIC smoke 還保留另一層不同性質的證據：在同一 shared state、same lineage 與相容 execution contract 下，獨立節點+GPU 路徑的 computation-state fingerprint 呈現**幾乎一致／高度一致**。

這一項不應與 Exact Transport SHA 混為一談。它代表的是：當 state 已進入各自的 node + GPU execution path 後，不同 worker 的運算指紋仍維持極高一致性；但因 GPU kernel、driver、precision、execution order 與後續 action branch 可能產生差異，本白皮書不把它寫成「任意 GPU 都 bitwise identical」。

因此 RC2 把這項證據獨立標示為：**Node + GPU Computation Fingerprint Near-Match**。它和 exact transport、progressive QREV、cross-environment recomputed SHA 一起，構成四種不同的 SHA／fingerprint 證據。

4. SHA 證據如何解讀

Q-Framework 使用三種不同的 SHA 語義：

A. Exact Transport SHA

state 在沒有被運算修改時：

$\text{SOURCE SHA} = \text{BRIDGE SHA} = \text{TARGET SHA}$

代表 transport integrity。

B. Progressive QREV SHA

GPU 真正對 state 做下一步 compute：

$\text{SHA}_n \neq \text{SHA}_{n+1}$

代表 state 已推進，而不是重播舊 block。

C. Cross-Environment Computed SHA

不同 node/runtime 對同一 production state 執行相同計算：

$\text{computed_SHA_A} = \text{computed_SHA_B} = \text{computed_SHA_C}$

代表 solver result consistency。

這三種驗證共同構成 Q-Framework 的可追溯 state lineage。

5. 目前仍不宣稱的事項

RC2 增加已驗證證據，但仍不宣稱：

- 無限 VRAM。
 - 零 OOM。
 - 任意模型皆能直接套用 Q-Framework。
 - 任意 GPU／driver／precision 組合都具有 universal bitwise determinism。
 - 22GB RTX 2080 Ti 已完成最新 full-length 311 aligned H3 global-temporal production PASS。
 - production streamed-QKV、page residency、eviction/prefetch policy 已公開。
-

6. 公開邊界

本白皮書公開：

- 架構角色與 state lifecycle。
- Smoke / runtime 結果。
- SHA、receipt、benchmark 方法。
- 可公開的 reference interface 與 sanitized proof logic。

本白皮書不公開：

- QmRNA production pressure scoring。
- adaptive feed equation / coefficient。
- page/residency selection。
- eviction / prefetch policy。
- Qvram production serializer / restore ordering。
- Qsearch ranking。
- scheduler weights / fencing / retry authority。
- production streamed-QKV data-plane。

Open Specification • Open Evidence • Open Reference Interfaces • Closed Production Engine